

Web性能手术刀：前端性能优化系统性讲解（下）

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 Web性能手术刀：前端性能优化系统性讲解（上）](#)

2. 简历

作为面试的敲门砖，大/中/小厂的简历分别有什么异同点？如果在简历面上胜人一筹？

2.1 简历内容详细分析

简历模板的选择很讲究，有些简历基本不看内容就会被刷掉，这些简历一般会对面试官进行视觉攻击，让简历给面试官的第一印象就是反感。

有两种**令人反感的简历模板**：

1. 一种是经典简历模板，真是堪称经典，配色比较魔幻，加上表格类的简历属于low到底端的简历类型，基本上扫一眼就扔了，这种简历只需要3秒钟就能被面试官扔到垃圾堆。
2. 另一种是设计感十足的简历模板，这种简历设计感十足，这五颜六色的配色能亮瞎面试官的双眼，这种花里胡哨的简历同样也是3秒钟沉到垃圾堆底部的简历。

以上两类简历模板堪称面试官杀手，只要用了上述两类模板，绝对连让面试官看第二眼的兴趣都没有。面试官筛简历要的是**高效、清晰、内容突出**，不管是HR还是技术面试官都想在最快速的情况下看到有效信息，你眼中所谓的『视觉效果』在别人眼里就是『视觉噪音』或者『视觉垃圾』，严重影响看简历的心情和寻找有效信息的速度。

2.1.1 准备个人信息

个人信息部分主要包括**姓名、电话、电子邮箱、求职意向**，当然这四个是必填的，其它的都是选填，填好了是加分项，否则很可能减分。

接下来才是重点：

- 1.**博客**：如果准备一个基本没有更新的博客或者没有任何贡献的github，那么给面试官一种为了放上去而放上去的感觉，这基本上就是在跟面试官说『**这个候选人平时根本没有总结提炼的习惯**』，所以如果有长期维护的github或者博客一定要放上去，质量好的话会非常有用，如果没有千万别放。
- 2.**学历**：如果你的学历是专科、高中毕业之类的，还写在简历头部强调一遍，这就增加了简历被刷的几率，如果是研究生或者985211学历可以写，突出一下学历优势，本科学历在技术面试领域基本上敲门砖级别的，没必要写。
- 3.**年龄**：如果是大龄程序员，尤其是还在求一份低端岗位的时候千万别写，一个大龄程序员在求职一个中低端岗位，说明这些年基本原地踏步，还不能加班，到这里基本上此简历就凉了一半了。

4.照片：形象优秀的可以贴，尤其是形象优秀的女程序媛，其它的最好不要贴，如果要贴的话，最好是贴那种PS过的非常职业的证件照。

2.1.2 准备专业技能

对于程序员的专业技能其实就是技术栈，对于自己的技术栈如何描述是个很难的问题，比如什么算是精通？什么算是了解？什么是熟悉？

关于对技术技能的描述有很多种，这里只讲三种的了解、熟悉、精通：

●**了解**：使用过某一项技术，能在别人指导下完成工作，但不能胜任复杂工作，也不能独立解决问题。

●**熟悉**：大量运用过的某一项技术，能独立完成工作，且能独立完成有一定复杂度的工作，在技术的应用层面不会有太大问题，甚至理解一点原理。

●**精通**：不仅可以运用某一门技术完成复杂项目，而且理解这项技术背后的原理，可以对此技术进行二次开发，甚至本身就是技术源码的贡献者。

以Vue框架为例，如果可以用Vue写简单的页面，单独完成某几个页面的开发，但是无法脱离公司脚手架工作，也无法独立从0完成一个有一定复杂度的项目，只能称之为了解。

如果有大量运用vue的经验，有从0独立完成一定复杂度项目的能力，可以完全脱离脚手架进行开发，且对vue的原理有一定的了解，可以称之为熟悉。

如果用vue完成过复杂度很高的项目，而且非常熟悉vue的原理，是vue源码的主要贡献者，亦或者根据vue源码进行过魔改（比如mpvue），可以称得上精通。

那么有两个坑是候选人经常犯的，**杂和精**，这种两个坑大量集中在应届生和刚毕业没两年的新手身上，其主要特点是急于表现自我、对技术深度与广度出现无知而导致的过度自信。

首先说杂，比如要应聘一个Vue前端，老老实实把自己的Vue技术栈写好就行了，强调一下自己擅长什么即可，最好专精某领域比如后台管理系统实践等。这时一些简历非要给自己加戏，自己会的不会的一股脑往上堆，什么工程化定制、各个系统从0到1落地、AI都要体现出来，越杂越好，这种简历给人的印象就是个什么都不懂的半吊子。

再说精，一个刚毕业的应届生，出来简历就各种精通，精通Vue、精通Vue源码、精通React源码等等，这种简历是不会没头没脑投过来的，这种在大学里就精通各种的天才早被他的各种学长介绍进了大厂或者外企做某某Star重点培养了，往往看到的这种也是半吊子。

2.1.3 准备工作经历

工作经历本身不用花太多笔墨去写，面试官主要想看的就是每段工作经历的持续时间、在不同公司担任的职责如何、是否有大厂的工作经验等。

那么什么简历在这里给面试官减分呢？

●**频繁跳槽**：比如三年换了三家公司，每个公司呆的时长不要超过一年；

●**常年初级岗**：比如工作五六年之后依然在完成一些简单的项目开发；

●**末流公司经历**：在技术招聘届，大厂的优先级最高比如BAT、TMD甚至微软、谷歌等外企，知名度独角兽其次，一般的互联网公司排在第三，就是工作中小型的互联网公司一般大家叫不上名字，排在最后的就是外包和传统企业的经历；

（1）所以，如果你有频繁跳槽的经历怎么办？在本公司老老实实等到满一年再跳槽。

（2）如果常年初级岗怎么办？想办法晋升或者参与一些业界知名项目，再或者写一个有一定复杂度的私人项目。

（3）如果有末流公司经历怎么办？如果是很久以前的末流公司经验可以直接不写，也没人在乎你很早之前的工作经历，如果你现在就在末流公司，赶紧想办法跳槽，去不了大厂，去非知名的互联网公司也算是胜利大逃亡了。

2.1.4 准备项目经历

项目经历不管对于社招还是校招都是重中之重，很多时候成败就在于项目经历这块，一个普通本科可以通过优秀的项目经历逆袭985，一个小厂的员工也可以获得大厂的面试机会。但是必须要说一下项目经历的编写很讲究，这是为后面面试部分铺路的绝佳机会，也是直接让你的简历扑街的重点沦陷区域。

先来看简历中项目经历的坑。

2.1.4.1 切忌流水账写法

项目经历流水账写法是绝大多数简历的通病，通篇下来就讲了一件事『我干了啥』。

大部分简历却是这样的：

用Vue、Vuex、Vue-router、axios等技术开发电商网站的前端部分，主要负责首页、店铺详情、商品详情、商品列表、订单详情、订单中心等相关页面的开发工作，与设计师与后端配合，可要高度还原设计稿。

这个描述有什么问题？其实看似也没啥问题，但是这种流水账写法太多了，完全无法突出自己的优势展现自己的能力。**项目经历是考察重点，面试官想知道候选人在一次项目经历中扮演的角色、负责的模块、碰到的问题、解决的思路、达成的效果以及最后的总结与沉淀。**

而上面的描述只显示了我干了啥，所以这种项目描述几乎是没意义的，因为对于面试官而言他看不到有效信息，没有有效信息的项目描述基本就没价值了，如果这个时候你还没有大厂经历或者名校背书，基本上也就凉了。

2.1.4.2 切忌堆积项目

堆积项目这种现象往往出现在没有什么优秀项目经历的简历身上，候选人企图以数量优势掩盖质量的劣势，其实往往适得其反，项目经历的一栏最好放2-3个项目，非常优秀的项目可能放一个就足够了，举个极端例子如果有一天尤雨溪写简历，其实只需要在项目经历那些一行Vue.js作者就行了,当然,他并不需要投简历。

有一些项目切忌放上去：

- demo级项目：很多简历居然还在放一些仿xx官网的demo，这是十足的减分项，有一些则是东拼西凑抄了一些框架的源码搞了个玩具项目，也没有任何价值。
- 烂大街的项目：这种以vue技术栈的为最，由于视频网站的某门课程流行，导致大量的仿饿了么、仿qq音乐、仿美团、仿去哪儿，十份简历5份一模一样的项目，你是面试官会怎么想。
- 低质量的开源项目：一个大原则就是低star的尽量别放（除非是高质量代码的冷门项目），长期弃坑的也不要放，不要为了凑数量把低质量的项目暴露出来，好好藏着。

如果只放两个项目，最好的搭配是一个公司内部挑大梁的项目和一个社区内的开源项目，后者之所以可以占据一席之地，是因为通过你的开源项目，面试官可以通过commit完整看到你的创造过程，比如工程化建设、commit规范、代码规范、协作方式、代码能力、沟通能力等等，这甚至比面试都有用，没有比开源项目更能展示你综合素质的东西了。

2.1.4.3 切忌放虚假项目

一个项目做没做过只要有经验的面试官一问便知，如果你真的靠假项目忽悠过了面试，那这个公司八成也有问题，人才把关不过硬，你可以想象你的队友都是什么水平，在这种公司大成长价值也不大。好，如果你说实在没项目可写了，我只能造假了，那么你应该想一下这多层追问。

比如你说你优化了一个前端项目的首屏性能，降低了白屏时间，那么面试官对这个性能优化问题会进行深挖，来考察候选人的实际水平：

- 你的性能优化指标是怎么确定的？平均下来时间减短了多少？
- 你的性能是如何测试的？有两种主流的性能测试方法你是怎么选的？
- 你是根据哪些指标进行针对性优化的？
- 除了你说的这些优化方法还有没有想过通过xx来解决？
- 你的这个优化方法在实际操作中碰到过什么问题吗？有没有进一步做过测试？
- 我们假设这么一种情况，比如xxxx，你会这么进行优化？

面试官多层追问的逻辑是这样的：**了解背景 -> 了解方案 -> 深挖方案 -> 模拟场景**

首先得了解你性能优化的指标如何，接着需要了解你是怎么测试的指标、再怎么进行针对性优化的，再接着提出一些其它解决方案考察你对优化场景的知识储备和方案决策能力，最后再模拟一个其它的业务场景，来考察你的技能迁移能力，看看是否是对某块领域有一定的了解，而不是只针对某个项目。

如果要真的在面试现场对答如流，那么一定是在某一块领域有一定知识储备的人，不是随随便便搞个项目就能蒙混过关的。

2.1.4.4 合格的项目经历

合格的项目经历必须要有以下几点：

- 项目概述
- 个人职责
- 项目难点
- 工作成果

如果不怕字太多，还可以选择性加入解决方案、选型思路等，但是由于篇幅限制和为面试铺垫就不太建议写得太多。

项目概述的目的是让面试官理解项目，不是每个人面试官都做过你的那种项目，所以需要个简述方便面试官理解。

个人职责就是告诉面试官你在本项目中扮演的角色，是领导者？主导者？还是跟随者，你负责了哪些模块，承担了多大的工作量，以此来评估你在团队中的作用。

项目难点的目的在于让面试官看到你碰到的技术难题，方便后续面试对项目进行一系列讨论。

工作成果就很明显了，面试官需要看到你在做了上述工作到底达成了什么成绩，这个时候最好以数据说话，比如访问量、白屏时间等等。

这个时候也切忌长篇大论，把技术细节都写上去，甚至还写了心路历程的都是大忌，一方面篇幅太大会造成视觉混乱，另一方面面试官想看到的是简历，不是技术总结，面试官要面对上百份简历没那么时间来看你长篇大论，长篇大论大可以在面试中展开。

最好的方法就是一行文字简单得说清楚即可，反正项目面的时候会问到，到时候把准备的内容讲给面试官，掌握面试的主动权就是从项目经历这一栏中开始。

2.1.4.5 其他

• 教育背景

应届生可以写得更详细一点，比如绩点排名怎么样，有没有突出的科目，社招就不要写太多了，简单的入学时间、学校、专业即可，而且写最高学历即可，没必要从初中就开始写学历流水账，没有人看的。

• 注意事项

自我评价不建议写：技术面试几乎没人看你的自我评价，连面试技术问题都嫌『talk is cheap show me the code』，你的自我评价除了占篇幅没啥用处，充其量算是面试官的干扰信息。

• 简历封面千万别搞

这都是一些简历制作网站骗用户付费的伎俩，不仅是互联网行业，其它行业我也没见过要简历封面这种无用操作的。

• 证书不建议写

应届生可以酌情考虑弄个六级证书什么的，对于社招而言，列一堆证书甚至是减分项，国内的各种证你也懂的，是有多不自信才沦落到靠一堆证书来证明自己的价值。

• 千万别用技能图表

首先用90分、80分来评价自己的技术本身就没有什么说服力，也不可能这么精准，而且什么是90分、什么是80根本就没有一个公论，所以用一般的比较通用的熟悉、精通描述即可，千万别加戏，面试官或者HR没那么多闲工夫去理解你的图表，老老实实按最通用高效的方式描述自己的技术栈。

- 简历最好不超过2页

程序员又不是设计师有时候需要作品呈现，如果简历超过一页一定是出问题了，项目、技术栈描述太多太杂占据了大量篇幅。

2.2 真实简历分析&优化

飞书链接：[目 13. 如何写出高质量的简历（For 付费学员）](#) 密码：86t&9493

3. 具体技术栈应该掌握到什么程度？

 印客2025Web前端大厂训练营.pdf
19.34MB



4. React性能优化

4.1 不必要组件更新

这类优化是在组件状态发生变更后，通过减少不必要的组件更新来实现，是本文优化技巧的主要部分。

4.1.1 PureComponent、React.memo

在 React 工作流中，如果只有父组件发生状态更新，即使父组件传给子组件的所有 Props 都没有修改，也会引起子组件的 Render 过程。从 React 的声明式设计理念来看，如果子组件的 Props 和 State 都没有改变，那么其生成的 DOM 结构和副作用也不应该发生改变。当子组件符合声明式设计理念时，就可以忽略子组件本次的 Render 过程。PureComponent 和 React.memo 就是应对这种场景的，PureComponent 是对类组件的 Props 和 State 进行浅比较，React.memo 是对函数组件的 Props 进行浅比较。

4.1.2 shouldComponentUpdate

在 React 刚开源的那段时期，数据不可变性还没有现在这样流行。当时 Flux 架构就使用的模块变量来维护 State，并在状态更新时直接修改该模块变量的属性值，而不是使用[展开语法](#)生成新的对象引用。例如要往数组中添加一项数据时，当时的代码很可能是 `state.push(item)`，而不是 `const newState = [...state, item]`。

在此背景下，当时的开发者经常使用 `shouldComponentUpdate` 来深比较 Props，只在 Props 有修改才执行组件的 Render 过程。如今由于数据不可变性和函数组件的流行，这样的优化场景已经不会再出现了。

接下来介绍另一种可以使用 `shouldComponentUpdate` 来优化的场景。在项目初始阶段，开发者往往图方便会给予子组件传递一个大对象作为 Props，后面子组件想用啥就用啥。当大对象中某个「子组件未使用的属性」发生了更新，子组件也会触发 Render 过程。在这种场景下，通过实现子组件的 `shouldComponentUpdate` 方法，仅在「子组件使用的属性」发生改变时才返回 `true`，便能避免子组件重新 Render。

但使用 `shouldComponentUpdate` 优化第二个场景有两个弊端。

1. 如果存在很多子孙组件，「找出所有子孙组件使用的属性」就会有很多工作量，也容易因为漏测导致 bug；
2. 存在潜在的工程隐患。举例来说，假设组件结构如下：

Code block

```
1  <A data="{data}">
2    { /* B 组件只使用了 data.a 和 data.b */ }
3    <B data="{data}">
4      { /* C 组件只使用了 data.a */ }
5      <C data="{data}"></C>
6    </B>
7  </A>
```

B 组件的 `shouldComponentUpdate` 中只比较了 `data.a` 和 `data.b`，目前是没任何问题的。之后开发者想在 C 组件中使用 `data.c`，假设项目中 `data.a` 和 `data.c` 是一起更新的，所以也没任何问题。但这份代码已经变得脆弱了，如果某次修改导致 `data.a` 和 `data.c` 不一起更新了，那么系统就会出问题。而且实际业务中代码往往更复杂，从 B 到 C 可能还有若干中间组件，这时就很难想到是 `shouldComponentUpdate` 引起的问题了。

4.1.3 useMemo、useCallback 实现稳定的 Props 值

如果传给子组件的派生状态或函数，每次都是新的引用，那么 `PureComponent` 和 `React.memo` 优化就会失效。所以需要使 用 `useMemo` 和 `useCallback` 来生成稳定值，并结合 `PureComponent` 或 `React.memo` 避免子组件重新 Render。

4.1.4 发布者订阅者跳过中间组件 Render 过程

React 推荐将公共数据放在所有「需要该状态的组件」的公共祖先上，但将状态放在公共祖先上后，该状态就需要层层向下传递，直到传递给使用该状态的组件为止。

每次状态的更新都会涉及中间组件的 Render 过程，但中间组件并不关心该状态，它的 Render 过程只负责将该状态再传给子组件。在这种场景下可以将状态用发布者订阅者模式维护，只有关心该状态的组件才去订阅该状态，不再需要中间组件传递该状态。当状态更新时，发布者发布数据更新消息，只有订阅者组件才会触发 Render 过程，中间组件不再执行 Render 过程。

只要是发布者订阅者模式的库，都可以进行该优化。比如：`redux`、`use-global-state`、`React.createContext` 等。本示例使用 `React.createContext` 进行实现。

<https://codesandbox.io/s/fabuzhedingyuezhemoshitiaoguozechongjianzujiande-render-guocheng-nm7nt?file=/src/PubSubCommunicate.js>

Code block

```
1  import { useState, useEffect, createContext, useContext } from "react"
2
3  const renderCntMap = {}
4  const renderOnce = name => {
5    return (renderCntMap[name] = (renderCntMap[name] || 0) + 1)
6  }
7
8  // 将需要公共访问的部分移动到 Context 中进行优化
9  // Context.Provider 就是发布者
10 // Context.Consumer 就是消费者
11 const ValueCtx = createContext()
12 const CtxContainer = ({ children }) => {
13   const [cnt, setCnt] = useState(0)
14   useEffect(() => {
15     const timer = window.setInterval(() => {
16       setCnt(v => v + 1)
17     }, 1000)
18     return () => clearInterval(timer)
19   }, [setCnt])
20
21   return <ValueCtx.Provider value={cnt}>{children}</ValueCtx.Provider>
22 }
23
24 function CompA({}) {
25   const cnt = useContext(ValueCtx)
26   // 组件内使用 cnt
27   return <div>组件 CompA Render 次数: {renderOnce("CompA")}</div>
28 }
29
30 function CompB({}) {
31   const cnt = useContext(ValueCtx)
32   // 组件内使用 cnt
33   return <div>组件 CompB Render 次数: {renderOnce("CompB")}</div>
34 }
35
```

```

36  function CompC({}) {
37    return <div>组件 CompC Render 次数: {renderOnce("CompC")}</div>
38  }
39
40  export const PubSubCommunicate = () => {
41    return (
42      <CtxContainer>
43        <div>
44          <h1>优化后场景</h1>
45          <div>
46            将状态提升至最低公共祖先的上层，用 CtxContainer 将其内容包裹。
47          </div>
48          <div style={{ marginTop: "20px" }}>
49            每次 Render 时，只有组件A和组件B会重新 Render 。
50          </div>
51
52          <div style={{ marginTop: "40px" }}>
53            父组件 Render 次数: {renderOnce("parent")}
54          </div>
55          <CompA />
56          <CompB />
57          <CompC />
58        </div>
59      </CtxContainer>
60    )
61  }
62
63  export default PubSubCommunicate
64

```

4.1.5 状态下放，缩小状态影响范围

如果一个状态只在某部分子树中使用，那么可以将这部分子树提取为组件，并将该状态移动到该组件内部。如下面的代码所示，虽然状态 color 只在 `<input />` 和 `<p />` 中使用，但 color 改变会引起 `<ExpensiveTree />` 重新 Render。

Code block

```

1  import { useState } from "react"
2
3  export default function App() {
4    let [color, setColor] = useState("red")
5    return (
6      <div>
7        <input value={color} onChange={e => setColor(e.target.value)} />
8        <p style={{ color }}>Hello, world!</p>
9        <ExpensiveTree />
10     </div>
11   )
12 }
13
14 function ExpensiveTree() {
15   let now = performance.now()
16   while (performance.now() - now < 100) {
17     // Artificial delay -- do nothing for 100ms
18   }
19   return <p>I am a very slow component tree.</p>
20 }
21

```

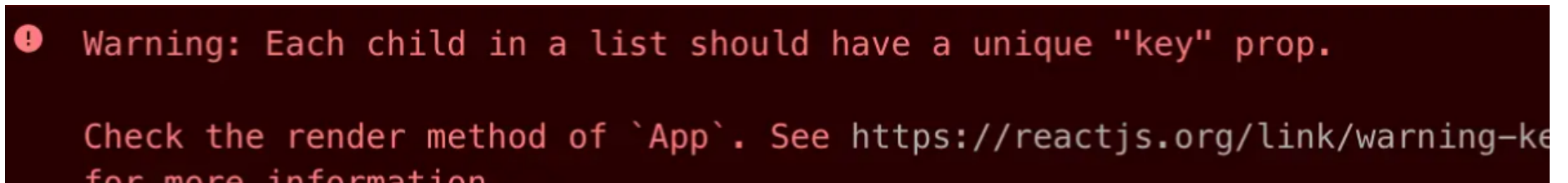
通过将 color 状态、`<input />` 和 `<p />` 提取到组件 Form 中，结果如下。

Code block

```
1  export default function App() {
2    return (
3      <>
4        <Form />
5        <ExpensiveTree />
6      </>
7    )
8  }
9
10 function Form() {
11   let [color, setColor] = useState("red")
12   return (
13     <>
14       <input value={color} onChange={e => setColor(e.target.value)} />
15       <p style={{ color }}>Hello, world!</p>
16     </>
17   )
18 }
19
```

4.1.6 列表项使用 key 属性

当渲染列表项时，如果不给组件设置不相等的属性 key，就会收到如下报警。



那 key 属性到底在优化了什么呢？，在不使用 key 时，组件两次 Render 的结果如下。

Code block

```
1  <!-- 前一次 Render 结果 -->
2  <ul>
3    <li>Duke</li>
4    <li>Villanova</li>
5  </ul>
6
7  <!-- 新的 Render 结果 -->
8  <ul>
9    <li>Connecticut</li>
10   <li>Duke</li>
11   <li>Villanova</li>
12 </ul>
```

此时 React 的 Diff 算法会按照 `<li>` 出现的先后顺序进行比较，得出结果为需要更新前两个 `<li>` 并创建内容为 Villanova 的 `li`，一共会执行两次 DOM 更新、一次 DOM 创建。

如果加上 React 的 key 属性，两次 Render 结果如下。

Code block

```
1  <!-- 前一次 Render 结果 -->
2  <ul>
3    <li key="2015">Duke</li>
4    <li key="2016">Villanova</li>
5  </ul>
```

```
6
7 <!-- 新的 Render 结果 -->
8 <ul>
9   <li key="2014">Connecticut</li>
10  <li key="2015">Duke</li>
11  <li key="2016">Villanova</li>
12 </ul>
```

React Diff 算法会把 key 值为 2015 的虚拟 DOM 进行比较，发现 key 为 2015 的虚拟 DOM 没有发生修改，不用更新。同样，key 值为 2016 的虚拟 DOM 也不需要更新。结果就只需要创建 key 值为 2014 的虚拟 DOM。相比于不使用 key 的代码，使用 key 节省了两次 DOM 更新操作。

4.1.7 useMemo 返回虚拟 DOM

利用 useMemo 可以缓存计算结果的特点，如果 useMemo 返回的是组件的虚拟 DOM，则将在 useMemo 依赖不变时，跳过组件的 Render 阶段。该方式与 React.memo 类似，但与 React.memo 相比有以下优势：

1. 更方便。React.memo 需要对组件进行一次包装，生成新的组件。而 useMemo 只需在存在性能瓶颈的地方使用，不用修改组件。
2. 更灵活。useMemo 不用考虑组件的所有 Props，而只需考虑当前场景中用到的值。

<https://codesandbox.io/s/usememo-tiaoguo-zujian-render-guocheng-bzz9r>

该例子中，父组件状态更新后，不使用 useMemo 的子组件会执行 Render 过程，而使用 useMemo 的子组件不会执行。

Code block

```
1  import { useEffect, useMemo, useState } from "react"
2  import "./styles.css"
3
4  const renderCntMap = {}
5
6  function Comp({ name }) {
7    renderCntMap[name] = (renderCntMap[name] || 0) + 1
8    return (
9      <div>
10        组件「{name}」 Render 次数: {renderCntMap[name]}
11      </div>
12    )
13  }
14
15  export default function App() {
16    const setCnt = useState(0)[1]
17    useEffect(() => {
18      const timer = window.setInterval(() => {
19        setCnt(v => v + 1)
20      }, 1000)
21      return () => clearInterval(timer)
22    }, [setCnt])
23
24    const comp = useMemo(() => {
25      return <Comp name="使用 useMemo 作为 children" />
26    }, [])
27
28    return (
29      <div className="App">
30        <Comp name="直接作为 children" />
31        {comp}
32      </div>
33    )
34  }
```

4.2 提交阶段优化

这类优化的目的是减少提交阶段耗时。

4.2.1 避免在 `didMount`、`didUpdate` 中更新组件 State

这个技巧不仅仅适用于 `didMount`、`didUpdate`，还包括 `willUnmount`、`useLayoutEffect` 和特殊场景下的 `useEffect`（当父组件的 `cDU/cDM` 触发时，子组件的 `useEffect` 会同步调用），本文为叙述方便将他们统称为「提交阶段钩子」。

React 工作流提交阶段的第二步就是执行提交阶段钩子，它们的执行会阻塞浏览器更新页面。如果在提交阶段钩子函数中更新组件 State，会再次触发组件的更新流程，造成两倍耗时。

一般在提交阶段的钩子中更新组件状态的场景有：

1. 计算并更新组件的派生状态（Derived State）。在该场景中，类组件应使用 `getDerivedStateFromProps` 钩子方法代替，函数组件应使用函数调用时执行 `setState` 的方式代替。使用上面两种方式后，React 会将新状态和派生状态在一次更新内完成。
2. 根据 DOM 信息，修改组件状态。在该场景中，除非想办法不依赖 DOM 信息，否则两次更新过程是少不了的，就只能用其他优化技巧了。

4.3 前端通用优化

4.3.1 批量更新

React 类组件中 `setState` 是同步的还是异步的？

答案是：在 React 管理的事件回调和生命周期中，`setState` 是异步的，而其他时候 `setState` 都是同步的。这个问题根本原因就是 React 在自己管理的事件回调和生命周期中，对于 `setState` 是批量更新的，而在其他时候是立即更新的。

批量更新 `setState` 时，多次执行 `setState` 只会触发一次 Render 过程。相反在立即更新 `setState` 时，每次 `setState` 都会触发一次 Render 过程，就存在性能影响。

假设有如下组件代码，该组件在 `getData()` 的 API 请求结果返回后，分别更新了两个 State。线上代码实操参考：

<https://codesandbox.io/s/batchupdates-pilianggengxin-qqdsc>

Code block

```
1  function NormalComponent() {
2    const [list, setList] = useState(null)
3    const [info, setInfo] = useState(null)
4
5    useEffect(() => {
6      ;(async () => {
7        const data = await getData()
8        setList(data.list)
9        setInfo(data.info)
10     })()
11   }, [])
12
13   return <div>非批量更新组件时 Render 次数: {renderOnce("normal")}</div>
14 }
15
```

该组件会在 `setList(data.list)` 后触发组件的 Render 过程，然后在 `setInfo(data.info)` 后再次触发 Render 过程，造成性能损失。遇到该问题，开发者有两种实现批量更新的方式来解决该问题：

1. 将多个 State 合并为单个 State。例如使用 `const [data, setData] = useState({ list: null, info: null })` 替代 `list` 和 `info` 两个 State。

2. 使用 React 官方提供的 `unstable_batchedUpdates` 方法，将多次 `setState` 封装到 `unstable_batchedUpdates` 回调中。修改后代码如下。

Code block

```
1  function BatchedComponent() {
2    const [list, setList] = useState(null)
3    const [info, setInfo] = useState(null)
4
5    useEffect(() => {
6      ;(async () => {
7        const data = await getData()
8        unstable_batchedUpdates(() => {
9          setList(data.list)
10         setInfo(data.info)
11       })
12     })()
13   }, [])
14
15   return <div>批量更新组件时 Render 次数: {renderOnce("batched")}</div>
16 }
17
```

4.3.2 按优先级更新，及时响应用户

优先级更新是批量更新的逆向操作，其思想是：优先响应用户行为，再完成耗时操作。

常见的场景是：页面弹出一个 Modal，当用户点击 Modal 中的确定按钮后，代码将执行两个操作。a) 关闭 Modal。b) 页面处理 Modal 传回的数据并展示给用户。当 b) 操作需要执行 500ms 时，用户会明显感觉到从点击按钮到 Modal 被关闭之间的延迟。

<https://codesandbox.io/s/youxianjigengxinlijixiangyingyonghucaozuo-eb740>

用户添加一个整数后，页面要隐藏输入框，并将新添加的整数加入到整数列表，将列表排序后再展示。以下为一般的实现方式，将 `slowHandle` 函数作为用户点击按钮的回调函数。

Code block

```
1  const slowHandle = () => {
2    setShowInput(false)
3    setNumbers([...numbers, +inputValue].sort((a, b) => a - b))
4  }
```

`slowHandle()` 执行过程耗时长，用户点击按钮后会明显感觉到页面卡顿。如果让页面优先隐藏输入框，用户便能立刻感知到页面更新，不会有卡顿感。**实现优先级更新的要点是将耗时任务移动到下一个宏任务中执行，优先响应用户行为。**例如在该例中，将 `setNumbers` 移动到 `setTimeout` 的回调中，用户点击按钮后便能立即看到输入框被隐藏，不会感知到页面卡顿。优化后的代码如下。

Code block

```
1  const fastHandle = () => {
2    // 优先响应用户行为
3    setShowInput(false)
4    // 将耗时任务移动到下一个宏任务执行
5    setTimeout(() => {
6      setNumbers([...numbers, +inputValue].sort((a, b) => a - b))
7    })
8  }
9
```

5. 组件设计优化 -- Intersection Observer

`IntersectionObserver` 接口（从属于 [Intersection Observer API](#)）提供了一种异步观察目标元素与其祖先元素或顶级文档视口（viewport）交叉状态的方法。其祖先元素或视口被称为根（root）。

当一个 `IntersectionObserver` 对象被创建时，其被配置为监听根中一段给定比例的可见区域。一旦 `IntersectionObserver` 被创建，则无法更改其配置，所以一个给定的观察者对象只能用来监听可见区域的特定变化值；然而，你可以在同一个观察者对象中配置监听多个目标元素。

caniuse: <https://caniuse.com/?search=Intersection%20Observer>

Code block

```
1  var observer = new IntersectionObserver(callback, options);
```

`IntersectionObserver` 支持两个参数：

- callback 是当被监听元素的可见性变化时，触发的回调函数
- options 是一个配置参数，可选，有默认的属性值

5.1 callback

目标元素的可见性变化时，就会调用观察器的回调函数callback。
callback一般会触发两次。一次是目标元素刚刚进入视口（开始可见），另一次是完全离开视口（开始不可见）。
接下来，就看一个官方的示例代码：

Code block

```
1  //初始化一个实例
2  var observer = new IntersectionObserver(changes => {
3      for (const change of changes) {
4          console.log(change.time);
5          // Timestamp when the change occurred
6          // 当可视状态变化时，状态发送改变的时间戳
7          // 对比时间为，实例化的时间，
8          // 比如，值为1000时，表示在IntersectionObserver实例化的1秒钟之后，触发该元素的可视性变化
9
10         console.log(change.rootBounds);
11         // Unclipped area of root
12         // 根元素的矩形区域信息，即为getBoundingClientRect方法返回的值
13
14         console.log(change.boundingClientRect);
15         // target.boundingBox()
16         // 目标元素的矩形区域的信息
17
18         console.log(change.intersectionRect);
19         // boundingClientRect, clipped by its containing block ancestors,
20         // and intersected with rootBounds
21         // 目标元素与视口（或根元素）的交叉区域的信息
22
23         console.log(change.intersectionRatio);
24         // Ratio of intersectionRect area to boundingClientRect area
25         // 目标元素的可见比例，即intersectionRect占boundingClientRect的比例，
26         // 完全可见时为1，完全不可见时小于等于0
27
28         console.log(change.target);
29         // the Element target
30         // 被观察的目标元素，是一个 DOM 节点对象
31         // 当前可视区域正在变化的元素
```

```
32
33     }
34 }, {}));
35
36 // Watch for intersection events on a specific target Element.
37 // 对元素target添加监听，当target元素变化时，就会触发上述的回调
38 observer.observe(target);
39
40 // Stop watching for intersection events on a specific target Element.
41 // 移除一个监听，移除之后，target元素的可视区域变化，将不再触发前面的回调函数
42 observer.unobserve(target);
43
44 // Stop observing threshold events on all target elements.
45 // 停止所有的监听
46 observer.disconnect();
```

上面代码中，回调函数采用的是箭头函数的写法。callback函数的参数（changes）是一个数组，每个成员都是一个 `IntersectionObserverEntry` 对象。举例来说，如果同时有两个被观察的对象的可见性发生变化，changes数组就会有两名成员。

5.1.1 IntersectionObserverEntry 对象

Code block

```
1  {
2    time: 3893.92,
3    rootBounds: ClientRect {
4      bottom: 920,
5      height: 1024,
6      left: 0,
7      right: 1024,
8      top: 0,
9      width: 920
10   },
11   boundingClientRect: ClientRect {
12     // ...
13   },
14   intersectionRect: ClientRect {
15     // ...
16   },
17   intersectionRatio: 0.54,
18   target: element
19 }
```

每个属性的含义如下：

time：可见性发生变化的时间，是一个高精度时间戳，单位为毫秒

target：被观察的目标元素，是一个 DOM 节点对象

rootBounds：根元素的矩形区域的信息，getBoundingClientRect()方法的返回值，如果没有根元素（即直接相对于视口滚动），则返回null

boundingClientRect：目标元素的矩形区域的信息

intersectionRect：目标元素与视口（或根元素）的交叉区域的信息

intersectionRatio：目标元素的可见比例，即intersectionRect占boundingClientRect的比例，完全可见时为1，完全不可见时小于等于0

5.2 Option 对象

`IntersectionObserver` 构造函数的第二个参数是一个配置对象。它可以设置以下属性。

5.2.1 threshold 属性

`threshold`属性决定了什么时候触发回调函数。它是一个数组，每个成员都是一个门槛值，默认为[0]，即交叉比例（`intersectionRatio`）达到0时触发回调函数。

Code block

```
1  new IntersectionObserver(  
2    entries => { /* ... */ },  
3    {  
4      threshold: [0, 0.25, 0.5, 0.75, 1]  
5    }  
6  );
```

用户可以自定义这个数组。比如，[0, 0.25, 0.5, 0.75, 1]就表示当目标元素 0%、25%、50%、75%、100% 可见时，会触发回调函数。

5.2.2 root 属性 & rootMargin 属性

很多时候，目标元素不仅会随着窗口滚动，还会在容器里面滚动（比如在iframe窗口里滚动）。容器内滚动也会影响目标元素的可见性。

`IntersectionObserver` API 支持容器内滚动。`root`属性指定目标元素所在的容器节点（即根元素）。注意，容器元素必须是目标元素的祖先节点。

Code block

```
1  var opts = {  
2    root: document.querySelector('.container'),  
3    rootMargin: "500px 0px"  
4  };  
5  
6  var observer = new IntersectionObserver(  
7    callback,  
8    opts  
9  );
```

上面代码中，除了`root`属性，还有`rootMargin`属性。后者定义根元素的margin，用来扩展或缩小`rootBounds`这个矩形的大小，从而影响`intersectionRect`交叉区域的大小。它使用CSS的定义方法，比如10px 20px 30px 40px，表示 top、right、bottom 和 left 四个方向的值。

这样设置以后，不管是窗口滚动或者容器内滚动，只要目标元素可见性变化，都会触发观察器

5.3 应用

5.3.1 懒加载（lazy load）

我们希望某些静态资源（比如图片），只有用户向下滚动，它们进入视口时才加载，这样可以节省带宽，提高网页性能。这就叫做"惰性加载"。

有了 `IntersectionObserver` API，实现起来就很容易了。

Code block

```
1  function query(selector) {  
2    return Array.from(document.querySelectorAll(selector));  
3  }  
4  
5  var observer = new IntersectionObserver(  
6    callback,  
7    {  
8      root: document.querySelector('.container'),  
9      rootMargin: "500px 0px"  
10   }  
11  );
```

```

6     function(changes) {
7         changes.forEach(function(change) {
8             var container = change.target;
9             var content = container.querySelector('template').content;
10            container.appendChild(content);
11            observer.unobserve(container);
12        });
13    }
14 );
15
16 query('.lazy-loaded').forEach(function (item) {
17     observer.observe(item);
18 });

```

上面代码中，只有目标区域可见时，才会将模板内容插入真实 DOM，从而引发静态资源的加载。

5.3.2 无限滚动

Code block

```

1  var intersectionObserver = new IntersectionObserver(
2      function (entries) {
3          // 如果不可见，就返回
4          if (entries[0].intersectionRatio <= 0) return;
5          loadItems(10);
6          console.log('Loaded new items');
7      });
8
9  // 开始观察
10 intersectionObserver.observe(
11     document.querySelector('.scrollerFooter')
12 );

```

5.4 核心Hooks实现

Code block

```

1  import { RefObject, useEffect, useState } from 'react';
2
3  interface Args extends IntersectionObserverInit {
4      freezeOnceVisible?: boolean;
5  }
6
7  export const useIntersectionObserver = (
8      elementRef: RefObject<Element>,
9      { threshold = 0, root = null, rootMargin = '0%', freezeOnceVisible = false }: Args,
10 ): IntersectionObserverEntry | undefined => {
11     const [entry, setEntry] = useState<IntersectionObserverEntry>();
12
13     const frozen = entry?.isIntersecting && freezeOnceVisible;
14
15     const updateEntry = ([entry]: IntersectionObserverEntry[]): void => {
16         setEntry(entry);
17     };
18
19     useEffect(() => {
20         const node = elementRef?.current; // DOM Ref
21         const hasIOSupport = !!window.IntersectionObserver;
22

```

```

23     if (!hasIOSupport || frozen || !node) return;
24
25     const observerParams = { threshold, root, rootMargin };
26     const observer = new IntersectionObserver(updateEntry, observerParams);
27
28     observer.observe(node);
29
30     return () => observer.disconnect();
31     // eslint-disable-next-line react-hooks/exhaustive-deps
32   }, [elementRef?.current, JSON.stringify(threshold), root, rootMargin, frozen]);
33
34   return entry;
35 };

```

6. 组件按需加载 -- babel-plugin-import

平时在使用 `antd`、`element` 等组件库的时候，都会使用到一个 `Babel` 插件：`babel-plugin-import`，那它是用来干什么，又是如何实现的呢？

6.1 Babel-plugin-import 介绍

6.1.1 为什么需要 babel-plugin-import

`antd` 和 `element` 这两个组件库，看它的源码，`index.js` 分别是这样的：

Code block

```

1  // antd
2  export { default as Button } from './button';
3  export { default as Table } from './table';

```

Code block

```

1  // element
2  import Button from '../packages/button/index.js';
3  import Table from '../packages/table/index.js';
4  export default {
5    Button,
6    Table,
7  };

```

`antd` 和 `element` 都是通过 `ES6 Module` 的 `export` 来导出带有命名的各个组件。

所以，我们可以通过 `ES6` 的 `import { } from` 的语法来导入单组件的 `JS` 文件。但是，我们还需要手动引入组件的样式：

Code block

```

1  // antd
2  import 'antd/dist/antd.css';
3  // element
4  import 'element-ui/lib/theme-chalk/index.css';

```

如果仅仅是只需要一个 `Button` 组件，却把所有的样式都引入了，这明显是不合理的。

当然也可以只使用单个组件，还可以减少代码体积：

Code block

```

1  import Button from 'antd/lib/button';

```

```
2   import 'antd/lib/button/style';
```

但是，看一下如们需要多个组件的时候：

Code block

```
1   import { Affix, Avatar, Button, Rate } from 'antd';
2
3   import 'antd/lib/affix/style';
4   import 'antd/lib/avatar/style';
5   import 'antd/lib/button/style';
6   import 'antd/lib/rate/style';
```

会不会觉得这样的代码不够优雅？

这时候就应该思考一下，如何在引入 `Button` 的时候自动引入它的样式文件。

6.1.2 Babel-plugin-import 做了什么

简单来说，`babel-plugin-import` 就是解决了上面的问题，为组件库实现单组件按需加载并且自动引入其样式，如：

Code block

```
1   import { Button } from 'antd';
2
3       ↓ ↓ ↓ ↓ ↓ ↓
4
5   var _button = require('antd/lib/button');
6   require('antd/lib/button/style');
```

只需关心需要引入哪些组件即可，内部样式我并不需要关心，你帮我自动引入就 ok。

6.1.3 这个插件怎么用

简单来说就需要关心三个参数即可：

Code block

```
1   {
2     "libraryName": "antd",      // 包名
3     "libraryDirectory": "lib",  // 目录，默认 lib
4     "style": true,              // 是否引入 style
5   }
```

其它的看文档：<https://github.com/umijs/babel-plugin-import#usage>

6.2 Babel-plugin-import 源码解析

主要来看一下 `babel-plugin-import` 如何加载 `JavaScript` 代码和样式的。

以下面这段代码为例：

Code block

```
1   import { Button, Rate } from 'antd';
2   ReactDOM.render(<Button>xxxx</Button>);
```

6.2.1 依赖收集

`babel-plugin-import` 会在 `ImportDeclaration` 里将所有的 `specifier` 收集起来。

需要做的事情也很简单：

1. `import` 的包是不是 `antd`，也就是 `libraryName`
2. 把 `Button` 和 `Rate` 收集起来

代码实现：

Code block

```
1  ImportDeclaration(path, state) {
2    const { node } = path;
3    if (!node) return;
4    // 代码里 import 的包名
5    const { value } = node.source;
6    // 配在插件 options 的包名
7    const { libraryName } = this;
8    // babel-type 工具函数
9    const { types } = this;
10   // 内部状态
11   const pluginState = this.getPluginState(state);
12   // 判断是不是需要使用该插件的包
13   if (value === libraryName) {
14     // node.specifiers 表示 import 了什么
15     node.specifiers.forEach(spec => {
16       // 判断是不是 ImportSpecifier 类型的节点，也就是是否是大括号的
17       if (types.isImportSpecifier(spec)) {
18         // 收集依赖
19         // 也就是 pluginState.specified.Button = Button
20         // local.name 是导入进来的别名，比如 import { Button as MyButton } from 'antd' 的 MyButton
21         // imported.name 是真实导出的变量名
22         pluginState.specified[spec.local.name] = spec.imported.name;
23       } else {
24         // ImportDefaultSpecifier 和 ImportNamespaceSpecifier
25         pluginState.libraryObjs[spec.local.name] = true;
26       }
27     });
28     pluginState.pathsToRemove.push(path);
29   }
30 }
```

待 `babel` 遍历了所有的 `ImportDeclaration` 类型的节点之后，就收集好了依赖关系，下一步就是如何加载它们了。

6.2.2 判断如何使用

收集了依赖关系之后，得要判断一下这些 `import` 的变量是否被使用到了，我们这里说一种情况。

我们知道，`JSX` 最终是变成 `React.createElement()` 执行的：

Code block

```
1  ReactDOM.render(<Button>Hello</Button>);
2
3      ↓ ↓ ↓ ↓ ↓ ↓
4
5  React.createElement(Button, null, "Hello");
```

没错，`createElement` 的第一个参数就是我们要找的东西，我们需要判断收集的依赖中是否有被 `createElement` 使用。

Code block

```

1  CallExpression(path, state) {
2    const { node } = path;
3    const file = (path && path.hub && path.hub.file) || (state && state.file);
4    // 方法调用者的 name
5    const { name } = node.callee;
6    // babel-type 工具函数
7    const { types } = this;
8    // 内部状态
9    const pluginState = this.getPluginState(state);
10
11    // 如果方法调用者是 Identifier 类型
12    if (types.isIdentifier(node.callee)) {
13      if (pluginState.specified[name]) {
14        node.callee = this.importMethod(pluginState.specified[name], file, pluginState);
15      }
16    }
17
18    // 遍历 arguments 找我们要的 specifier
19    node.arguments = node.arguments.map(arg => {
20      const { name: argName } = arg;
21      if (
22        pluginState.specified[argName] &&
23        path.scope.hasBinding(argName) &&
24        path.scope.getBinding(argName).path.type === 'ImportSpecifier'
25      ) {
26        // 找到 specifier, 调用 importMethod 方法
27        return this.importMethod(pluginState.specified[argName], file, pluginState);
28      }
29      return arg;
30    });
31  }

```

除了 `React.createElement(Button)` 之外，还有 `const btn = Button / [Button]` ... 等多种情况会使用 `Button`，源码中都有对应的处理方法：<https://github.com/umijs/babel-plugin-import/blob/master/src/Plugin.js#L177>

6.2.3 生成引入代码

第一步和第二步主要的工作是找到需要被插件处理的依赖关系，比如：

Code block

```

1  import { Button, Rate } from 'antd';
2  ReactDOM.render(<Button>Hello</Button>);

```

`Button` 组件使用到了，`Rate` 在代码里未使用。所以插件要做的也只是自动引入 `Button` 的代码和样式即可。

我们先回顾一下，当我们 `import` 一个组件的时候，希望它能够：

Code block

```

1  import { Button } from 'antd';
2
3      ↓ ↓ ↓ ↓ ↓ ↓
4
5  var _button = require('antd/lib/button');
6  require('antd/lib/button/style');

```

并且再回想一下插件的配置 `options`，只需要将 `libraryDirectory` 以及 `style` 等配置用上就可以了。

但是，这里该如何让 `babel` 去修改代码并且生成一个新的 `import` 以及一个样式的 `import` 呢？

Code block

```
1  import { addSideEffect, addDefault, addNamed } from '@babel/helper-module-imports';
2
3  importMethod(methodName, file, pluginState) {
4    if (!pluginState.selectedMethods[methodName]) {
5      // libraryDirectory: 目录, 默认 lib
6      // style: 是否引入样式
7      const { style, libraryDirectory } = this;
8
9      // 组件名转换规则
10     // 优先级最高的是配了 camel2UnderlineComponentName: 是否使用下划线作为连接符
11     // camel2DashComponentName 为 true, 会转换成小写字母, 并且使用 - 作为连接符
12     const transformedMethodName = this.camel2UnderlineComponentName
13       ? transCamel(methodName, '_')
14       : this.camel2DashComponentName
15       ? transCamel(methodName, '-')
16       : methodName;
17     // 兼容 windows 路径
18     // path.join('antd/lib/button') == 'antd/lib/button'
19     const path = winPath(
20       this.customName
21       ? this.customName(transformedMethodName, file)
22       : join(this.libraryName, libraryDirectory, transformedMethodName, this.fileName),
23     );
24     // 根据是否有导出 default 来判断使用哪种方法来生成 import 语句, 默认为 true
25     // addDefault(path, 'antd/lib/button', { nameHint: 'button' })
26     // addNamed(path, 'button', 'antd/lib/button')
27     pluginState.selectedMethods[methodName] = this.transformToDefaultImport
28       ? addDefault(file.path, path, { nameHint: methodName })
29       : addNamed(file.path, methodName, path);
30     // 根据不同配置 import 样式
31     if (this.customStyleName) {
32       const stylePath = winPath(this.customStyleName(transformedMethodName));
33       addSideEffect(file.path, `${stylePath}`);
34     } else if (this.styleLibraryDirectory) {
35       const stylePath = winPath(
36         join(this.libraryName, this.styleLibraryDirectory, transformedMethodName, this.fileName),
37       );
38       addSideEffect(file.path, `${stylePath}`);
39     } else if (style === true) {
40       addSideEffect(file.path, `${path}/style`);
41     } else if (style === 'css') {
42       addSideEffect(file.path, `${path}/style/css`);
43     } else if (typeof style === 'function') {
44       const stylePath = style(path, file);
45       if (stylePath) {
46         addSideEffect(file.path, stylePath);
47       }
48     }
49   }
50   return { ...pluginState.selectedMethods[methodName] };
51 }
```

`addSideEffect`, `addDefault` 和 `addNamed` 是 `@babel/helper-module-imports` 的三个方法, 作用都是创建一个 `import` 方法, 具体表现是:

6.2.3.1 addSideEffect

Code block

```
1  addSideEffect(path, 'source');
2
3      ↓ ↓ ↓ ↓ ↓ ↓
4
5  import "source"
```

6.2.3.2 addDefault

Code block

```
1  addDefault(path, 'source', { nameHint: "hintedName" })
2
3      ↓ ↓ ↓ ↓ ↓ ↓
4
5  import hintedName from "source"
```

6.2.3.3 addNamed

Code block

```
1  addNamed(path, 'named', 'source', { nameHint: "hintedName" });
2
3      ↓ ↓ ↓ ↓ ↓ ↓
4
5  import { named as _hintedName } from "source"
```

更多关于 `@babel/helper-module-imports` 见: [@babel/helper-module-imports](#)

6.2.4 总结

总结一下，`babel-plugin-import` 和普遍的 `babel` 插件一样，会遍历代码的 `ast`，然后在 `ast` 上做了一些事情：

- 1. **收集依赖**：找到 `importDeclaration`，分析出包 `a` 和依赖 `b,c,d....`，假如 `a` 和 `libraryName` 一致，就将 `b,c,d...` 在内部收集起来
- 2. **判断是否使用**：在多种情况下（比如文中提到的 `CallExpression`）判断 收集到的 `b,c,d...` 是否在代码中被使用，如果有使用的，就调用 `importMethod` 生成新的 `import` 语句
- 3. **生成引入代码**：根据配置项生成代码和样式的 `import` 语句

到这一步为止，可以发现，其实除了 `antd` 和 `element` 等大型组件库之外，任意的组件库都可以使用 `babel-plugin-import` 来实现按需加载和自动加载样式。

比如我们常用的 `lodash`，也可以使用 `babel-plugin-import` 来加载它的各种方法，可以动手试一下。

6.3 手动实现babel-plugin-import

6.3.1 最简功能实现

按照上文说的，最重要的配置项就是三个：

Code block

```
1  {
2    "libraryName": "antd",
3    "libraryDirectory": "lib",
4    "style": true,
5  }
```

所以我们也就只实现这三个配置项。

并且，上文提到，真实情况中会有多种方式来调用一个组件，这里我们也不处理这些复杂情况，只实现最常见的 `<Button />` 调用。

6.3.2 入口文件

入口文件的作用是获取用户传入的配置项并且将核心插件代码作用到 `ast` 上。

Code block

```
1  import Plugin from './Plugin';
2
3  export default function ({ types }) {
4    let plugins = null;
5
6    // 将插件作用到节点上
7    function applyInstance(method, args, context) {
8      for (const plugin of plugins) {
9        if (plugin[method]) {
10          plugin[method].apply(plugin, [...args, context]);
11        }
12      }
13    }
14
15    const Program = {
16      // ast 入口
17      enter(path, { opts = {} }) {
18        // 初始化插件实例
19        if (!plugins) {
20          plugins = [
21            new Plugin(
22              opts.libraryName,
23              opts.libraryDirectory,
24              opts.style,
25              types,
26            ),
27          ];
28        }
29        applyInstance('ProgramEnter', arguments, this);
30      },
31      // ast 出口
32      exit() {
33        applyInstance('ProgramExit', arguments, this);
34      },
35    };
36
37    const ret = {
38      visitor: { Program },
39    };
40
41    // 插件只作用在 ImportDeclaration 和 CallExpression 上
42    ['ImportDeclaration', 'CallExpression'].forEach(method => {
43      ret.visitor[method] = function () {
44        applyInstance(method, arguments, ret.visitor);
45      };
46    });
47
48    return ret;
49  }
```

6.3.3 核心代码

真正修改 `ast` 的代码是在 `plugin` 实现的：

Code block

```
1  import { join } from 'path';
2  import { addSideEffect, addDefault } from '@babel/helper-module-imports';
3
4  /**
5   * 转换成小写，添加连接符
6   * @param {*} _str 字符串
7   * @param {*} symbol 连接符
8   */
9  function transCamel(_str, symbol) {
10     const str = _str[0].toLowerCase() + _str.substr(1);
11     return str.replace(/([A-Z])/g, $1 => `${symbol}${$1.toLowerCase()}`);
12 }
13
14 /**
15 * 兼容 Windows 路径
16 * @param {*} path
17 */
18 function winPath(path) {
19     return path.replace(/\\/g, '/');
20 }
21
22 export default class Plugin {
23     constructor(
24         libraryName, // 需要使用按需加载的包名
25         libraryDirectory = 'lib', // 按需加载的目录
26         style = false, // 是否加载样式
27         types, // babel-type 工具函数
28     ) {
29         this.libraryName = libraryName;
30         this.libraryDirectory = libraryDirectory;
31         this.style = style;
32         this.types = types;
33     }
34
35     /**
36     * 获取内部状态，收集依赖
37     * @param {*} state
38     */
39     getPluginState(state) {
40         if (!state) {
41             state = {};
42         }
43         return state;
44     }
45
46     /**
47     * 生成 import 语句 (核心代码)
48     * @param {*} methodName
49     * @param {*} file
50     * @param {*} pluginState
51     */
52     importMethod(methodName, file, pluginState) {
53         if (!pluginState.selectedMethods[methodName]) {
54             // libraryDirectory: 目录，默认 lib
55             // style: 是否引入样式
56             const { style, libraryDirectory } = this;
57             // 组件名转换规则
58             const transformedMethodName = transCamel(methodName, '');
59         }
60     }
61 }
```

```

59      // 兼容 windows 路径
60      // path.join('antd/lib/button') == 'antd/lib/button'
61      const path = winPath(join(this.libraryName, libraryDirectory, transformedMethodName));
62      // 生成 import 语句
63      // import Button from 'antd/lib/button'
64      pluginState.selectedMethods[methodName] = addDefault(file.path, path, { nameHint: methodName });
65      if (style) {
66          // 生成样式 import 语句
67          // import 'antd/lib/button/style'
68          addSideEffect(file.path, `${path}/style`);
69      }
70  }
71  return { ...pluginState.selectedMethods[methodName] };
72  }
73
74  ProgramEnter(path, state) {
75      const pluginState = this.getPluginState(state);
76      pluginState.specified = Object.create(null);
77      pluginState.selectedMethods = Object.create(null);
78      pluginState.pathsToRemove = [];
79  }
80
81  ProgramExit(path, state) {
82      // 删除旧的 import
83      this.getPluginState(state).pathsToRemove.forEach(p => !p.removed && p.remove());
84  }
85
86  /**
87   * ImportDeclaration 节点的处理方法
88   * @param {*} path
89   * @param {*} state

```

7. vue优化

7.1 更新优化

非必要不更新

改变状态方式：props、state

7.2 Props 稳定性

在 Vue 之中，一个子组件只会在其至少一个 props 改变时才会更新。思考以下示例：

Code block

```

1  <ListItem
2    v-for="item in list"
3    :id="item.id"
4    :active-id="activeId" />

```

在 `<ListItem>` 组件中，它使用了 `id` 和 `activeId` 两个 props 来确定它是否是当前活跃的那一项。虽然这是可行的，但问题是每当 `activeId` 更新时，列表中的每一个 `<ListItem>` 都会跟着更新！

理想情况下，只有活跃状态发生改变的项才应该更新。我们可以将活跃状态比对的逻辑移入父组件来实现这一点，然后让 `<ListItem>` 改为接收一个 `active` prop：

Code block

```

1  <ListItem
2    v-for="item in list"

```

```
3      :id="item.id"
4      :active="item.id === activeId" />
```

7.3 v-once

shouldComponentUpdate

`v-once` 是一个内置的指令，可以用来渲染依赖运行时数据但无需再更新的内容。它的整个子树都会在未来的更新中被跳过。在随后的重新渲染，元素/组件及其所有子项将被当作静态内容并跳过渲染。这可以用来优化更新时的性能。

Code block

```
1  <!-- 单个元素 -->
2  <span v-once>This will never change: {{msg}}</span>
3  <!-- 带有子元素的元素 -->
4  <div v-once>
5    <h1>comment</h1>
6    <p>{{msg}}</p>
7  </div>
8  <!-- 组件 -->
9  <MyComponent v-once :comment="msg" />
10 <!-- `v-for` 指令 -->
11 <ul>
12   <li v-for="i in list" v-once>{{i}}</li>
13 </ul>
```

7.4 v-memo

`v-memo` 是一个内置指令，可以用来有条件地跳过某些大型子树或者 `v-for` 列表的更新。

缓存一个模板的子树。在元素和组件上都可以使用。为了实现缓存，该指令需要传入一个固定长度的依赖值数组进行比较。如果数组里的每个值都与最后一次的渲染相同，那么整个子树的更新将被跳过。举例来说：

template

Code block

```
1  <div v-memo="[valueA, valueB]">
2    ...
3  </div>
```

当组件重新渲染，如果 `valueA` 和 `valueB` 都保持不变，这个 `<div>` 及其子项的所有更新都将被跳过。实际上，甚至虚拟 DOM 的 vnode 创建也将被跳过，因为缓存的子树副本可以被重新使用。

正确指定缓存数组很重要，否则应该生效的更新可能被跳过。`v-memo` 传入空依赖数组 (`v-memo="[]"`) 将与 `v-once` 效果相同。

与 `v-for` 一起使用

`v-memo` 仅用于性能至上场景中的微小优化，应该很少需要。最常见的情况可能是有助于渲染海量 `v-for` 列表 (长度超过 1000 的情况)：

template

Code block

```
1  <div v-for="item in list" :key="item.id" v-memo="[item.id === selected]">
2    <p>ID: {{ item.id }} - selected: {{ item.id === selected }}</p>
3    <p>...more child nodes</p>
4  </div>
```

当组件的 `selected` 状态改变，默认会重新创建大量的 vnode，尽管绝大部分都跟之前是一模一样的。`v-memo` 用在这里本质上是在说“只有当该项的被选中状态改变时才需要更新”。这使得每个选中状态没有变的项能完全重用之前的 vnode 并跳过差异比较。注意这里 memo 依赖数组中并不需要包含 `item.id`，因为 Vue 也会根据 item 的 `:key` 进行判断。

8. 师资&课程内容



印客2025Web前端大厂训练营.pdf
19.34MB



8.1 项目实战内容

github: <https://github.com/encode-studio-fe>

[目前端实战课分享汇总](#)（可咨询班主任要密码权限）

8.2 服务内容

系统&突击两手抓

全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



8.3 专业一对一

近期一对一

- [\(2025-02-06\) xxx-2.5年-React-24K](#)
- [\(2025-02-07\) xxx-1年-React-15k](#)

• 国（2025-01-05）xxx-7年-Vue-20k

报名学员专享

目 13. 如何写出高质量的简历（For 付费学员）

8.4 渠道内推 & 保障就业

渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

8.5 近期报名学员案例

